

DAT42-2

Coding & Working with LLMs

Calling APIs, prompting, embeddings, and failure modes

Using LLMs as practical tools from Python.

Albert School — 16 hours

Contents

1	What an LLM does, practically	2
2	Calling LLM APIs from Python	2
2.1	Request and response structure	3
2.2	Error handling and retry logic	3
2.3	Token counting and cost estimation	3
3	Prompt engineering	4
4	Systematic prompt evaluation	4
5	Embeddings and semantic similarity	4
6	LLM failure modes	5
6.1	When LLMs are inappropriate	5
7	Cost-effectiveness	5

1 What an LLM does, practically

An LLM (large language model) predicts the next token given the tokens before it. A token is a chunk of text (roughly a word or word-piece). To generate text, the model repeatedly predicts the next token and appends it to the sequence.

Two parameters control how the next token is sampled from the model's predicted distribution:

- **Temperature** scales the distribution. A temperature near 0 makes the model almost always pick the most probable token (deterministic, repetitive); a higher temperature (e.g. 1.0) flattens the distribution, making less probable tokens more likely (more varied, less predictable).
- **Top-p** (nucleus sampling) restricts sampling to the smallest set of tokens whose cumulative probability reaches p . With `top_p = 0.9`, only the most probable tokens summing to 90% of the mass are considered.

This is the entire conceptual basis needed to use the API. How the model is built and trained is out of scope (see Math S4 and DAT62-1).

2 Calling LLM APIs from Python

You assume the Python prerequisites from DAT12-2: functions, error handling with `try/except`, installing libraries with `pip`, and parsing JSON responses.

An LLM API is called like any other web API: you send a request describing the conversation and receive a response. Use the official SDK for the provider. The examples below use Anthropic's API; OpenAI's is structurally equivalent.

```
pip install anthropic
import anthropic

client = anthropic.Anthropic() # reads ANTHROPIC_API_KEY from the environment

response = client.messages.create(
    model="claude-opus-4-8",
    max_tokens=1024,
    messages=[
        {"role": "user", "content": "What is the capital of France?"}
    ],
)
print(response.content[0].text)
```

2.1 Request and response structure

- **Request:** the model identifier, a `max_tokens` limit on the output, and a list of `messages`. Each message has a `role` ("user" or "assistant") and `content`. The conversation history is sent on every call — the API is stateless.
- **Response:** an object containing the generated `content` and a `usage` field reporting how many tokens were consumed.

2.2 Error handling and retry logic

API calls can fail: invalid input, authentication errors, rate limits, or temporary server errors. Rate-limit (HTTP 429) and server errors (HTTP 5xx) are **transient** and should be retried; client errors (4xx other than 429) should not. Retry transient errors with **exponential backoff** — wait a growing delay between attempts.

```
import time
import anthropic

client = anthropic.Anthropic()

def call_with_retry(messages, max_retries=5):
    for attempt in range(max_retries):
        try:
            return client.messages.create(
                model="claude-opus-4-8",
                max_tokens=1024,
                messages=messages,
            )
        except (anthropic.RateLimitError, anthropic.InternalServerError):
            if attempt == max_retries - 1:
                raise
            time.sleep(2 ** attempt) # 1s, 2s, 4s, 8s, ...
```

The official SDKs already retry these errors automatically with backoff; the snippet shows the logic the SDK applies.

2.3 Token counting and cost estimation

Usage is billed per token, separately for input and output. Count tokens before sending to estimate cost:

```
count = client.messages.count_tokens(
    model="claude-opus-4-8",
    messages=[{"role": "user", "content": "Hello"}],
)
print(count.input_tokens)
```

After a call, `response.usage` reports `input_tokens` and `output_tokens` actually consumed.

Cost is the token count multiplied by the per-token price. Prices are quoted per million tokens (MTok) and differ by model and by input vs. output. For example, at input \$5.00 / MTok and output \$25.00 / MTok, a call with 1,000 input tokens and 500 output tokens costs:

$$\frac{1000}{1000000} \times \$5.00 + \frac{500}{1000000} \times \$25.00 = \$0.0175.$$

A more capable model costs more per token; a smaller model costs less. Choosing the cheapest model that does the job is the main lever for controlling cost (see Section 7).

3 Prompt engineering

The prompt is the text you send the model. Prompt engineering is the craft of writing prompts that produce the output you want. The main techniques:

- **Zero-shot**: state the task directly, with no examples.

Classify the sentiment of this review as positive or negative: "The food was cold."

- **Few-shot**: include a few worked examples before the actual input, so the model infers the pattern.

Review: "Loved it!" -> positive

Review: "Terrible service." -> negative

Review: "The food was cold." ->

- **Chain-of-thought**: instruct the model to reason step by step before answering. This improves accuracy on multi-step problems.

A shop sells pens at 3 for \$2. How much do 12 pens cost? Think step by step, then give the answer.

Prompt structuring techniques shape the form of the output:

- **Persona assignment**: tell the model what role to adopt ("You are an expert tax accountant.").
- **Output format specification**: state the exact format required ("Reply with valid JSON only, with keys 'sentiment' and 'confidence'.").
- **Constraint injection**: add explicit constraints ("Answer in at most two sentences. Do not mention prices.").

4 Systematic prompt evaluation

Comparing prompting strategies by eyeballing one or two outputs is unreliable. Evaluate systematically:

1. Define a set of **criteria** the output must satisfy (e.g. correctness, format adherence, conciseness, absence of hallucination).
2. Build a **rubric**: for each criterion, define what counts as acceptable.
3. Run each prompting strategy on the same set of inputs.
4. Score each output against the rubric.
5. Compare the strategies on their scores.

The deliverable is a comparative evaluation of two prompting strategies (for example, zero-shot vs. few-shot) on the same task, scored against the same rubric, with a conclusion stating which performed better and why.

5 Embeddings and semantic similarity

An **embedding** is a fixed-length vector of numbers that represents a piece of text. A sentence embedding model maps a sentence to such a vector so that texts with similar meaning have vectors that are close together.

Closeness is measured by **cosine similarity**, the cosine of the angle between two vectors $\vec{u}$ and $\vec{v}$:

$$\text{cosine}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|}.$$

It ranges from -1 (opposite) through 0 (unrelated) to 1 (same direction, i.e. maximally similar in meaning).

```
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2")
u, v = model.encode(["a dog on the beach", "a puppy by the sea"])
cosine = u @ v / (np.linalg.norm(u) * np.linalg.norm(v))
print(cosine) # close to 1: the two sentences mean nearly the same thing
```

Basic semantic search: to find which of several documents is most relevant to a query, embed the query and every document, then return the documents with the highest cosine similarity to the query.

6 LLM failure modes

LLMs fail in characteristic ways. Know them, so you can tell when an LLM is the wrong tool.

- **Hallucination:** the model produces fluent, confident text that is factually wrong or fabricated (e.g. invented citations). The output looks correct but is not grounded in any source.
- **Sycophancy:** the model tends to agree with the user and tell them what they seem to want to hear, rather than what is correct.
- **Prompt injection:** untrusted text fed into the prompt (e.g. a web page or document the model is asked to summarise) contains instructions that hijack the model's behaviour. This is a security risk whenever the prompt includes content from outside your control.
- **Context window limits:** the model can only attend to a bounded number of tokens (the context window). Input beyond that limit cannot be processed — it must be truncated, split, or summarised, and information may be lost.

6.1 When LLMs are inappropriate

LLMs are a poor choice when:

- the task requires guaranteed factual accuracy and there is no way to verify the output (hallucination risk);
- the prompt must incorporate untrusted external text and the action taken is sensitive (prompt-injection risk);
- the input is larger than the context window and cannot be meaningfully split;
- a simpler, cheaper, deterministic method (a lookup, a regular expression, a rule) would solve the task reliably.

7 Cost-effectiveness

Justify LLM usage by comparing its cost and reliability against the alternative. An LLM call is **cost-effective** when the task genuinely needs language understanding or generation, the volume is manageable, and no cheaper deterministic method suffices. It is **wasteful** when a rule, lookup, or smaller model would do the same job — or when the failure modes in Section 6 make the output untrustworthy for the use at hand.

Estimate the cost (token count $\times$ price, summed over the expected volume), weigh it against the value produced and the cost of errors, and choose the cheapest model and method that meet the requirement.